

Nexus Gourmet

Documento de Arquitetura

Versão 1.0

Tabela - Integrantes do Grupo

Matrícula	Nome	Função (responsabilidade)	Pontos de participação na elaboração
242004457	Alexandre Henrique Almeida Valadares Sousa	Banco de dados	10
242028655	Davi Kenichi Watanabe Sakai	Frontend	10
241025953	Igor Lima Carneiro	Backend	10
242005329	Jhonatan William Araújo de Almeida	Banco de dados	10
242015432	João Gabriel Rolim Veiga	Backend	10
241039322	João Paulo Jacomini Batista	Frontend	10
241039304	João Victor Amorim Kurihara	Frontend	10
242024253	Lucas Ferreira Santana	Backend	10
242024271	Lucas Peixoto Rodrigues	Backend	10
242005006	Rafael de Aquino Marinho	Frontend	10

Histórico de Revisões

Data	Versão	Descrição	Autor(es)
14/05/2026	1.0	Primeira versão do documento que define a arquitetura usada no produto	Grupo Dijkstra

Sumário

1 Introdução.....	5
1.1 Propósito.....	5
1.2 Escopo.....	5
2 Representação Arquitetural.....	6
2.1 Definições.....	6
2.2 Justifique sua escolha.....	6
2.3 Detalhamento.....	7
Tabela 2 - Responsabilidades por camada.....	9
2.4 Metas e restrições arquiteturais.....	9
2.5 Visões.....	10
2.5.1 Visão de uso (o escopo do sistema).....	10
Fonte: Elaborado pelos autores (2026).....	11
2.5.2 Visão de organização lógica.....	11
2.5.3 Visão estrutural.....	12
2.6 Visão de Implantação.....	12
2.7 Restrições adicionais.....	12
3 Bibliografia.....	12

1 Introdução

1.1 Propósito

Este documento descreve a arquitetura do sistema sendo desenvolvido pelo grupo Dijkstra, na disciplina de MDS – Métodos de Desenvolvimento de Software – edição do primeiro semestre de 2026, para o sistema Nexus Gourmet, a fim de fornecer uma visão abrangente do sistema para desenvolvedores, testadores e demais interessados em aspectos relacionados às tecnologias a serem usadas no desenvolvimento.

1.2 Escopo

O detalhamento do escopo se encontra no documento de arquitetura, este, juntamente com o documento de Visão do produto e do projeto. Porém, em linhas gerais o escopo do produto compreende o desenvolvimento de um software capaz de registrar e organizar pedidos em restaurantes, como apresentado mais detalhadamente na seguinte tabela:

Tabela 1 - Funcionalidades presentes e não presentes

O que ele faz	o que ele não faz
Abrir pedido para mesa	Compra e venda de produtos
Adicionar item ao pedido	Adicionar conta de consumidor
Remover item do pedido	Permitir acesso direto de consumidores
Enviar pedido para a cozinha	
Visualizar pedidos na cozinha	
Visualizar tempo de espera	
Atualizar status do pedido	
Fechar conta	
Calcular total do pedido	
Visualizar mesas	
Cadastrar produtos	
Editar produto	

Fonte: elaborado pelo autor (2026)

2 Representação Arquitetural

2.1 Definições

O sistema Nexus Gourmet seguirá uma arquitetura Cliente-Servidor Web, organizada internamente segundo o padrão MVC em camadas, com apoio de comunicação assíncrona para atualização do status dos pedidos.

A escolha da arquitetura Cliente-Servidor ocorre porque o sistema será acessado por diferentes tipos de clientes, como dispositivos móveis utilizados pelos garçons, telas utilizadas pela cozinha e interfaces administrativas. Esses clientes consomem os serviços oferecidos por uma aplicação central, responsável por processar as regras de negócio, controlar o fluxo dos pedidos e acessar o banco de dados.

Internamente, o servidor será organizado segundo o padrão MVC (Model-View-Controller). Nesse modelo, a camada View representa as interfaces do sistema; a camada Controller recebe as requisições dos usuários e coordena o fluxo das operações; e a camada Model representa os dados, regras de negócio e persistência relacionados a pedidos, mesas, produtos, usuários e status de atendimento.

Além disso, como o Nexus Gourmet exige atualização rápida entre salão e cozinha, especialmente no envio e acompanhamento dos pedidos, a arquitetura prevê o uso de comunicação assíncrona, como WebSocket ou mecanismo equivalente, para notificar alterações de status sem depender exclusivamente de carregamento manual das páginas.

2.2 Justifique sua escolha

A escolha da arquitetura Cliente-Servidor Web com organização interna MVC é adequada ao Nexus Gourmet porque o produto proposto não é um sistema isolado em uma única máquina, mas uma aplicação distribuída entre diferentes usuários e dispositivos. O documento de Visão do Produto define o Nexus Gourmet como uma aplicação web-mobile voltada a proprietários e funcionários de restaurantes, com a finalidade de registrar, acompanhar e gerenciar pedidos de forma centralizada, ágil e sem erros.

O problema central identificado no projeto está relacionado à falha de comunicação entre salão e cozinha, à perda de informações em comandas de papel e à falta de rastreabilidade dos pedidos. Por isso, a arquitetura precisa favorecer a centralização das informações, o acesso simultâneo por múltiplos perfis de usuário e a atualização rápida do estado dos pedidos. A organização Cliente-Servidor atende diretamente a essa necessidade, pois separa os dispositivos consumidores dos serviços da aplicação central, mantendo o processamento e os dados em um servidor comum.

Essa escolha é adequada ao Nexus Gourmet porque o sistema possui características típicas de uma aplicação distribuída e modular, na qual diferentes usuários interagem simultaneamente com uma aplicação centralizada. O modelo Cliente-Servidor permite separar claramente os dispositivos de acesso da camada responsável pelo processamento das regras de negócio e armazenamento dos dados, favorecendo a organização, controle e sincronização das operações realizadas no restaurante.

No contexto do Nexus Gourmet, essa separação é importante porque o sistema será utilizado por diferentes perfis, como garçons, cozinha e administradores, cada um acessando funcionalidades específicas por meio de navegadores ou dispositivos conectados à aplicação

principal. Dessa forma, a centralização do processamento e das informações garante maior consistência no gerenciamento de pedidos, mesas, produtos e status de atendimento.

Para complementar essa estrutura, o sistema adotará o padrão MVC (Model-View-Controller) como organização interna da aplicação. Essa abordagem favorece a separação de responsabilidades entre interface, controle do fluxo da aplicação e manipulação dos dados, tornando o sistema mais organizado e compreensível.

A camada View será responsável pela interação com os usuários, apresentando telas e funcionalidades voltadas ao gerenciamento dos pedidos e operações do restaurante. A camada Controller coordenará o fluxo das requisições e regras de negócio, intermediando a comunicação entre interface e persistência. Já a camada Model concentrará as entidades e dados relacionados ao domínio do sistema, como pedidos, mesas, produtos e usuários.

Essa organização contribui diretamente para a manutenção e evolução do software, reduzindo o acoplamento entre partes do sistema e facilitando alterações futuras sem impacto excessivo em outros módulos. Além disso, a divisão clara das responsabilidades melhora o desenvolvimento paralelo entre as equipes de frontend, backend e banco de dados, permitindo maior produtividade e facilidade de integração.

Outro fator relevante é a necessidade de atualização rápida das informações entre salão e cozinha. Como o sistema exige acompanhamento contínuo dos pedidos e alteração dinâmica de status, a arquitetura também prevê comunicação assíncrona entre cliente e servidor, permitindo que mudanças importantes sejam refletidas em tempo próximo ao real. Isso possibilita maior agilidade operacional, reduz atrasos no atendimento e melhora a sincronização entre os diferentes setores do restaurante.

Assim, a combinação entre Cliente-Servidor, MVC e comunicação assíncrona oferece uma solução adequada às necessidades funcionais e estruturais do Nexus Gourmet, equilibrando organização, modularidade, manutenção, escalabilidade e eficiência operacional.

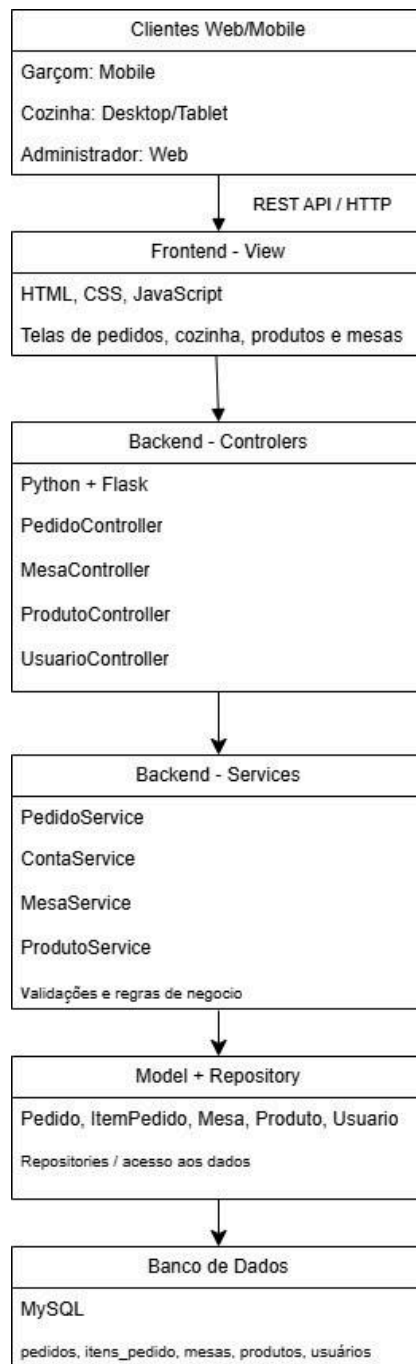
2.3 Detalhamento

A arquitetura proposta pode ser representada em quatro partes principais:

- **Clientes Web - Representam os dispositivos usados pelos perfis do sistema**
 - Garçom: abre pedidos, adiciona/remove itens, envia pedidos para a cozinha e fecha contas.
 - Cozinha: visualiza pedidos ativos, acompanha tempo de espera e atualiza status.
 - Administrador: cadastra e edita produtos, mesas e demais dados necessários à operação.
- **Camada de Apresentação — View**
 - Corresponde às páginas e interfaces desenvolvidas em HTML, CSS e JavaScript. Essa camada é responsável por apresentar as telas ao usuário e capturar suas ações, como clicar em “enviar pedido”, “adicionar item” ou “marcar como pronto”.
- **Camada de Controle e Aplicação — Controller/Service**
 - Corresponde ao backend da aplicação, desenvolvido em Python com Flask. Essa camada recebe as requisições das interfaces, valida as operações,

coordena o fluxo de dados e aciona as regras de negócio. É nela que ficam os controladores e serviços relacionados a pedidos, mesas, produtos, usuários e status.

- **Camada de Modelo e Persistência — Model/Repository/Database**
 - Representa os dados e regras centrais do sistema. Inclui as entidades principais, como Pedido, ItemPedido, Mesa, Produto, Usuário e StatusPedido. Também inclui os repositórios responsáveis por acessar o banco de dados MySQL, garantindo que os dados dos pedidos, produtos e mesas sejam



armazenados de forma consistente

Figura 1 - estilo arquitetural

Fonte: elaborado pelos autores (2026)

O fluxo principal começa quando o garçom acessa a interface web/mobile e registra um pedido vinculado a uma mesa. A View envia a solicitação ao Controller por meio de HTTP/HTTPS. O Controller aciona o Service correspondente, que valida as regras de negócio, como existência da mesa, disponibilidade do produto e cálculo dos valores. Em seguida, o Repository persiste os dados no MySQL.

Quando o pedido é enviado para a cozinha, o sistema altera seu status e disponibiliza essa alteração para a tela da cozinha. Para atender à necessidade de atualização em tempo real, a aplicação pode utilizar WebSocket ou outro mecanismo assíncrono, permitindo que a cozinha receba a atualização sem depender de recarregamento manual da página.

Quando o cozinheiro altera o status do pedido para “em preparo” ou “pronto”, o fluxo ocorre no sentido inverso: a View da cozinha envia a atualização ao Controller, o Service valida a mudança de estado e o banco de dados é atualizado. A interface do garçom pode então visualizar o novo status do pedido.

Tabela 2 - Responsabilidades por camada

Elemento	Responsabilidade no Nexus Gourmet
Cliente Web/Mobile	Permitir interação dos usuários com o sistema em diferentes dispositivos
View	Exibir telas, formulários, botões, listas de pedidos, mesas e produtos
Controller	Receber requisições, coordenar o fluxo e encaminhar ações aos serviços
Service	Aplicar regras de negócio, validações e cálculos
Model	Representar as entidades principais do domínio
Repository	Isolar o acesso ao banco de dados
Banco de Dados	Persistir pedidos, produtos, mesas, usuários e histórico de status
Comunicação assíncrona	Atualizar cozinha e salão quando houver mudança relevante no pedido

Fonte: elaborado pelos autores (2026)

2.4 Metas e restrições arquiteturais

Esta seção define metas e restrições arquiteturais que o sistema deve seguir.

- **Desempenho de Interface:** As atualizações de status de um pedido devem refletir na tela da cozinha.

- **Confiabilidade de Dados:** O sistema deve garantir que dados de pedidos não sejam perdidos caso haja alguma queda momentânea na conexão de internet.
- **Restrições de Rede:** O software precisa ser otimizado para operar na rede local do estabelecimento, a fim de minimizar a latência no envio dos pedidos do salão para a cozinha.
- **Escalabilidade e Concorrência:** O sistema está restrito a operar suportando múltiplos usuários (garçons e cozinheiros) logados de forma simultânea, especialmente em horários de pico.
- **Restrição de Segurança/Acesso:** É obrigatória a identificação (login) de qualquer funcionário para que o sistema permita a realização de operações de pedido.
- **Usabilidade Física:** Devido ao contexto dos garçons, o design da interface no dispositivo móvel tem a meta de possibilitar a operação e uso com apenas uma das mãos.
- **Métrica de Qualidade de Código:** O sistema tem como meta técnica manter uma densidade de erros de programa máxima de 0.5.%

2.5 Visões

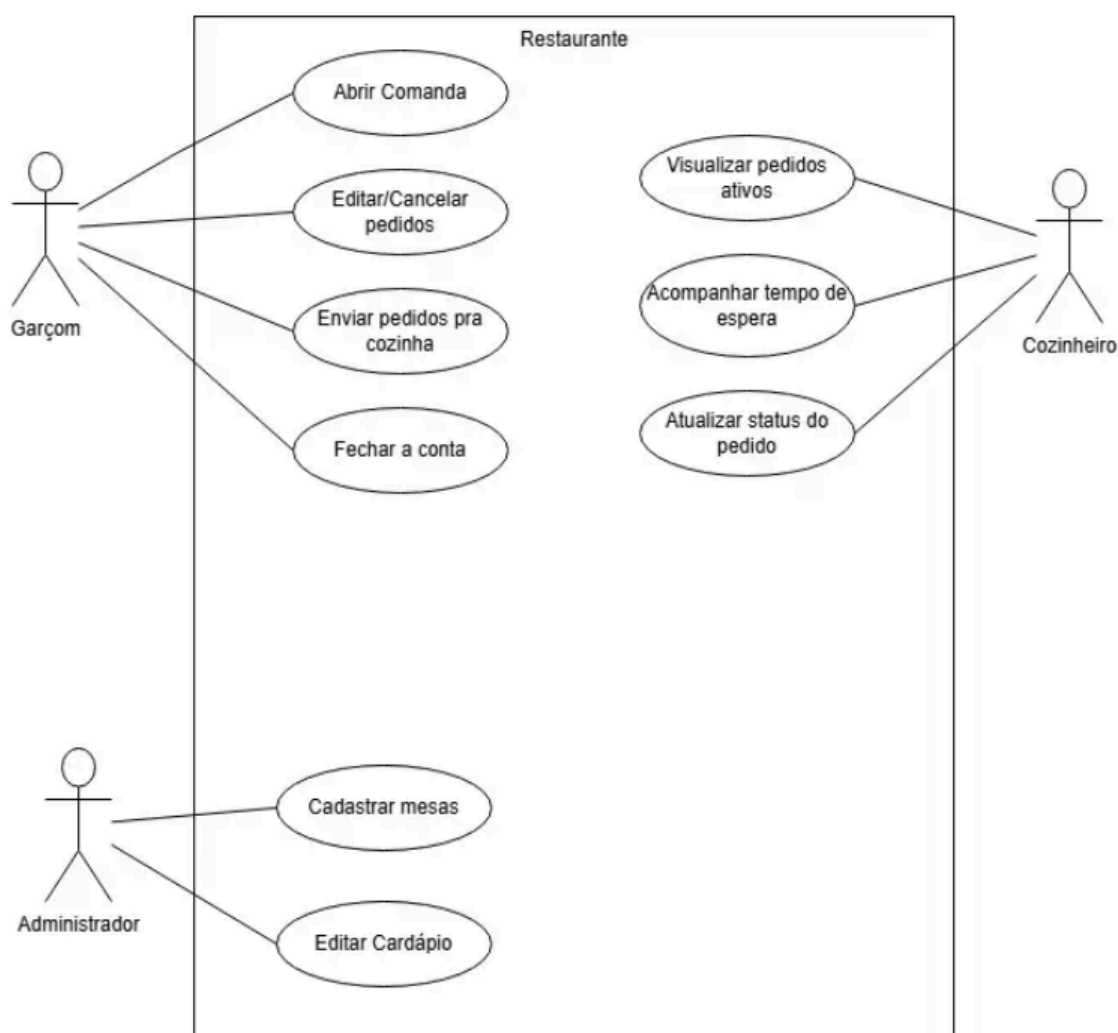
2.5.1 Visão de uso (o escopo do sistema)

O escopo do sistema Nexus Gourmet abrange o desenvolvimento de uma aplicação distribuída, com interfaces web e mobile, destinada à centralização e otimização do registro e gerenciamento de pedidos em ambientes gastronômicos. O fluxo operacional da aplicação engloba o ciclo completo do atendimento: a abertura da mesa e a inserção de itens pelo garçom; o envio simultâneo do pedido para a interface da cozinha, o monitoramento do tempo de preparo e a atualização de status ("em preparo" ou "pronto") pela equipe de cozinheiros, e, por fim, o encerramento da conta da mesa.

A definição do estilo arquitetural Cliente-Servidor Web, aliada ao padrão estrutural Model-View-Controller (MVC), foi fundamentada primordialmente nos requisitos operacionais do ambiente de implantação. A necessidade de mitigar falhas de comunicação históricas entre o salão e a cozinha exigiu uma arquitetura que garantisse a centralização das informações e o acesso simultâneo. Adicionalmente, a pluralidade de perfis de acesso sistêmico caracterizada pelo uso de dispositivos móveis pelos garçons, telas ou tablets na cozinha e interfaces web exclusivas para a administração justificou a separação lógica das interfaces gráficas (camada View) do processamento das regras de negócio e da persistência de dados.

Por fim, o requisito de atualização sistêmica ágil para o acompanhamento dos pedidos determinou a adoção de mecanismos de comunicação assíncrona, a exemplo do protocolo WebSocket, possibilitando que as mudanças de status sejam refletidas nas telas de operação sem a necessidade de recarregamento manual.

Figura 2 - Diagrama de Casos de Uso



Fonte: Elaborado pelos autores (2026)

2.5.2 Visão de organização lógica

O sistema é subdividido nos seguintes módulos.

- **Módulo de Apresentação (View):**
 - **Composição:** Desenvolvido com HTML, CSS e JavaScript.
 - **Razão Lógica:** É responsável unicamente pela interface com o usuário, apresentando as diferentes telas (mobile para garçom, desktop para cozinha) e capturando ações como "enviar pedido" ou "marcar como pronto".
- **Módulo de Controle e Serviços (Controller/Service):**
 - **Composição:** Desenvolvido em linguagem Python utilizando o framework Flask.
 - **Razão Lógica:** Coordena todo o fluxo da aplicação. Este módulo recebe as requisições das telas, efetua as devidas validações, aplica regras de negócio (cálculos de contas, existência da mesa) e determina as ações para persistência.

- **Módulo de Modelo e Persistência (Model/Repository/Database):**

- **Composição:** Composto pelas entidades fundamentais do sistema e o banco de dados MySQL.
- **Razão Lógica:** Este módulo serve para modelar o domínio (Pedido, ItemPedido, Mesa, Produto e Usuário) e garantir a integridade e isolamento na persistência física dos dados.

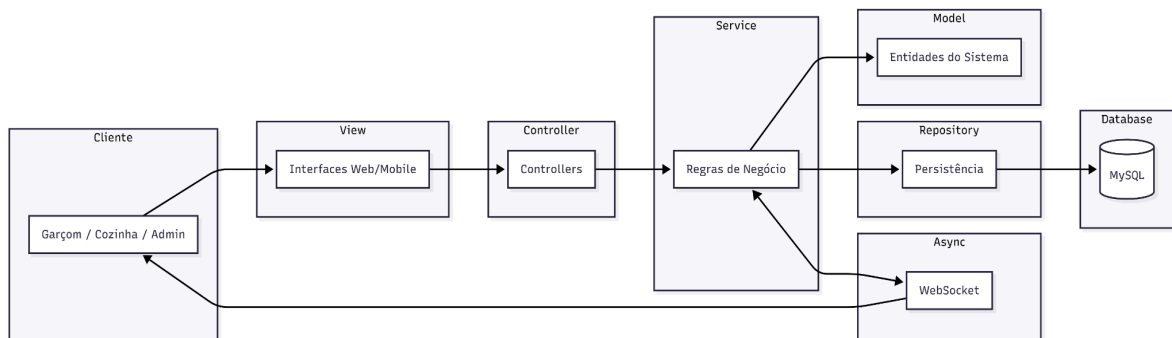
Como eles se comunicam (Interfaces):

A camada de Apresentação (View) comunica-se com os Controladores (backend) através de API REST utilizando o protocolo HTTP/HTTPS.

Para garantir o fluxo reverso dinâmico e em tempo real (como a cozinha notificando o garçom de que um prato finalizou), utiliza-se conexão assíncrona (WebSockets) entre a camada de Serviço e as Views.

Internamente, o Controller utiliza a camada de Service (regras de negócio) para se comunicar de maneira contínua com o Model e os Repositories, que por sua vez persistem e consultam as informações diretamente no banco de dados MySQL.

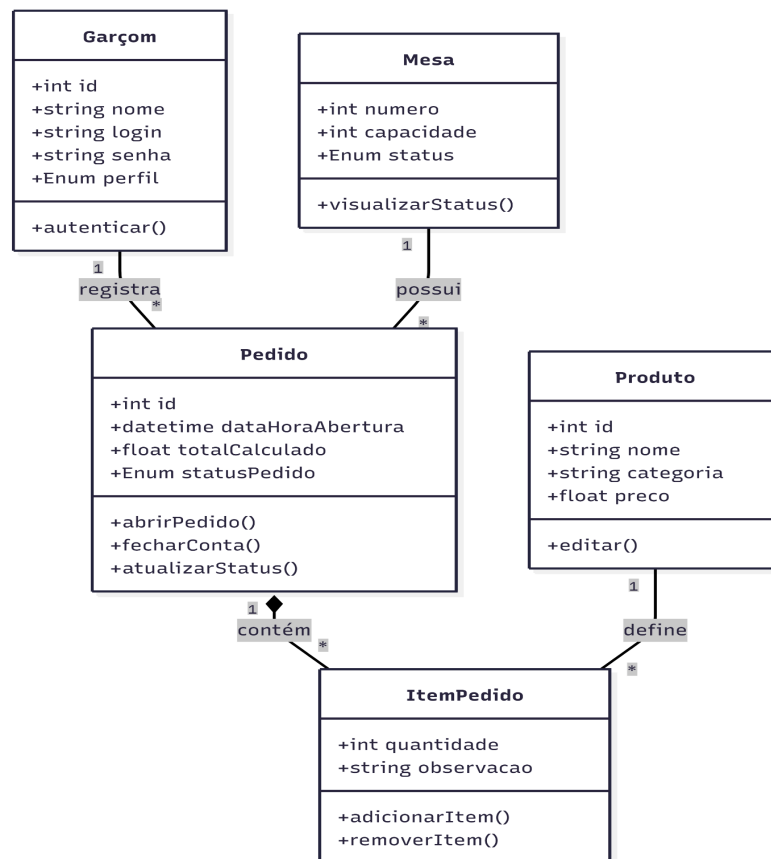
Figura 3 - Diagrama de Pacotes



Fonte: Elaborado pelos autores (2026)

2.5.3 Visão estrutural

Figura 4 - Diagrama de Classes



Fonte: Elaborado pelos autores (2026)

A Figura 4 ilustra o Diagrama de Classes, delimitando o escopo do produto e as interações entre os atores externos e as funcionalidades do sistema.

Detalhamento dos Atores e Interações:

- **Atores Primários:** O diagrama identifica três perfis distintos com responsabilidades segregadas: **Administrador**, **Garçon** e **Cozinheiro**. Essa separação é refletida na arquitetura de segurança e controle de acesso do sistema.
- **Gestão de Pedidos (Garçon/Cozinheiro):** O núcleo operacional do sistema é representado pelos casos de uso "Fazer pedido", "Visualizar pedidos" e "Atualizar status do pedido". A interação entre esses atores através do sistema mitiga falhas de comunicação e agiliza o tempo de atendimento.
- **Gestão Administrativa (Administrador):** O ator administrador possui permissões exclusivas para a manutenção do ecossistema, incluindo "Gerenciar mesas", "Gerenciar produtos" e "Gerenciar usuários", garantindo a integridade dos dados cadastrais utilizados nas operações de salão.

- **Fechamento de Ciclo:** O caso de uso "Fechar conta" encerra o fluxo operacional, integrando as seleções feitas pelo garçom com o cálculo financeiro final, conforme as regras de negócio estabelecidas.

Relevância Arquitetural:

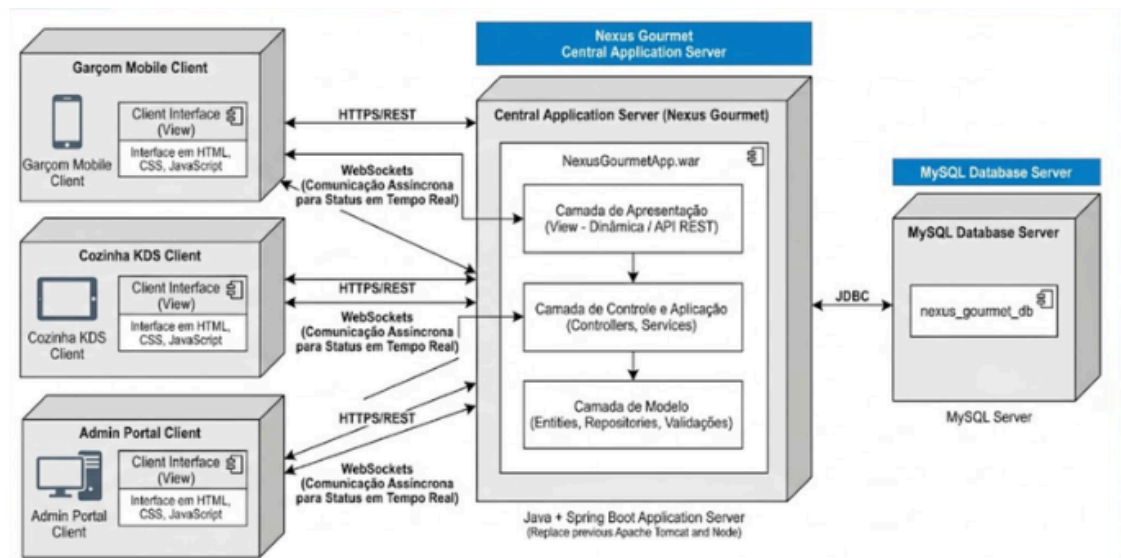
O mapeamento da Figura 4 serve como base para a definição das rotas da API no backend e para a construção das interfaces no frontend. Cada caso de uso representado foi priorizado para atender aos requisitos funcionais e garantir que a arquitetura lógica suporte a carga de trabalho simultânea de múltiplos atores.

2.6 Visão de Implantação

Descreve como o sistema será distribuído fisicamente, focando no modelo Cliente-Servidor para garantir a sincronização em tempo real entre salão e cozinha.

- **Dispositivos Clientes:**
 - **Interface Mobile:** O processo inicia com a autenticação do usuário. Após o login, o garçom executa as atividades de seleção de mesa e montagem do pedido. A atividade "Confirmar/Enviar" representa o gatilho de integração, onde os dados são despachados via API para o processamento central.
 - **Interface Desktop:** Esta partição descreve o ciclo de vida operacional na cozinha. O sistema KDS (Kitchen Display System) recebe o pedido e gerencia as atividades de "Preparação" e "Marcar Pronto". Esta transição é crítica para garantir a redução da carga cognitiva e eliminar o uso de papel.
 - **Entrega & Fechamento :** Descreve as atividades finais do ciclo de atendimento. A "Notificação/Retirada" serve como ponte de retorno para o salão, seguida pela "Entrega na Mesa" e a atividade financeira de "Pagamento/Fim", que culmina na liberação de recursos do sistema.
- **Banco de Dados:** O banco de dados será isolado da camada de aplicação para maior segurança, usando um sistema de gerenciamento relacional para garantir a consistência dos produtos cadastrados, além do registro de transações financeiras e pedidos.
- **Conectores:** A comunicação entre frontend e backend será feita via Protocolo HTTP.

Figura 5 - Diagrama de Implantação



Fonte: Elaborado pelos autores (2026)

2.7 Restrições adicionais

Esta seção detalha limitações e requisitos de qualidade que o sistema deve seguir.

- **Aspectos negociais:**
 - O software será acessível diretamente pela Internet e otimizado para a rede local do estabelecimento para evitar latência no envio de pedidos para cozinha.
 - O produto exigirá obrigatoriamente a identificação e login do usuário (funcionário do restaurante) para qualquer operação de pedido.
 - O sistema estará preparado para atender múltiplos usuários logados simultaneamente durante horários de pico do restaurante.
- **Característica de Qualidade de Software:**
 - **Confiabilidade:** Caso haja uma queda momentânea de conexão, o sistema não deve perder dados de pedidos.
 - **Usabilidade:** A interface seguirá padrões de design que possibilitem o uso com apenas uma mão, no caso dos garçons, facilitando o trabalho deles.

3 Bibliografia

IBM. **Modelo cliente/servidor**. IBM Documentation, [s. d.]. Disponível em: <https://www.ibm.com/docs/pt-br/cics-ts/5.6.0?topic=programs-clientserver-model>. Acesso em: 14 maio 2026.

SERRANO, Milene. **Arquitetura de Software: Visão Geral**. [S.l.]: Engenharia de Software UnB/FGA, 2026. 1 arquivo (70 slides), PDF.

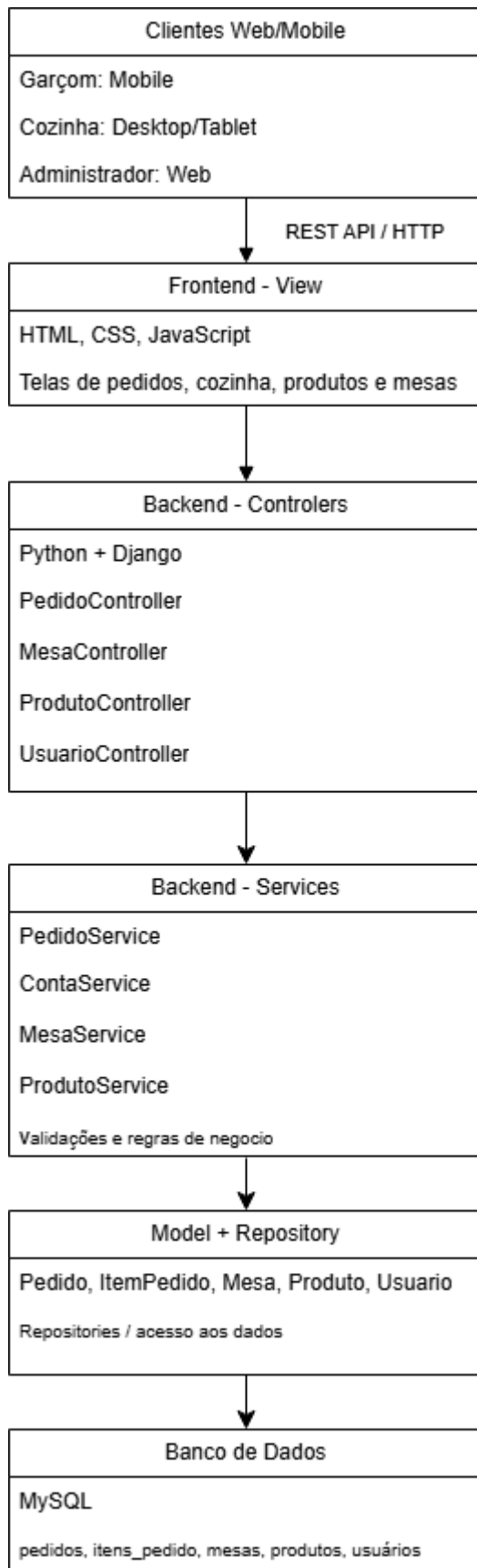


Diagrama para por no lugar da figura 1, na secão 2.3.